Exercise 1 — Student Management

Create a `Student` class with attributes: `studentId` (String), `fullName`, `className`, `averageScore` (double). - Use `HashMap<String, Student>` to store the list of students (key = student ID). - Write a program that allows: 1. Adding a new student. 2. Finding a student by ID. 3. Updating a student's average score. 4. Removing a student by ID. 5. Printing all students with a score >= 8.0.

Exercise 2 — Word Frequency Counter

Input a block of text (which may span multiple lines). - Count how many times each word appears (case-insensitive). - Use `HashMap<String, Integer>` to store the results (key = word, value = number of occurrences). - Print the 5 most frequent words (in descending order).

Exercise 3 — Warehouse Inventory Management

Create a `Product` class with: `productId`, `productName`, `stockQuantity`, `sellingPrice`. - Use `HashMap<String, Product>` (key = product ID). - Build a menu that allows: 1. Receiving stock (increases stock quantity). 2. Shipping stock out (decreases stock quantity, checking that enough stock is available). 3. Searching for a product by name (may return multiple results). 4. Printing the list of products that are low on stock (stock quantity less than 10). 5. Calculating the total value of all inventory.

Exercise 4 — Shopping Cart Management

Use `HashMap<String, CartItem>` (key = product ID). - Each `CartItem` contains: a product plus a quantity. - Write the following functions: 1. Add a product to the cart (if it's already there, increase the quantity). 2. Decrease the quantity of, or remove, a product from the cart. 3. Calculate the total cost of the cart. 4. Print the cart's details (sorted by product name).

Exercise 5 — Employee Salary Directory

Requirements: Write a Java program using a HashMap to manage employee salaries, where: - The key is the employee's name (String). - The value is the employee's salary (Integer/double).

The program should: 1. Add employee data (name and salary), for at least 6 employees. 2. Search for an employee's salary by their name; if not found, print "Employee not found". 3. Update the salary for a given name (bonus). 4. Remove an employee from the HashMap. 5. Print all employees and their salaries.

Exercise 6 — Contact List

Implement a simple contact list where names (keys) are stored with phone numbers (values) in a HashMap. - Write functions to add, remove, and search for contacts by name. - Extend the implementation to allow duplicate phone numbers under different names. - Bonus: Sort the HashMap entries by key (name) and display the sorted contact list.

Example:

```
HashMap<String, String> contacts = new HashMap<>();
contacts.put("Alice", "123-4567");
contacts.put("Bob", "987-6543");

// Searching by key (name):
System.out.println(contacts.get("Alice"));  // Output: 123-4567

// Removing a contact:
contacts.remove("Bob");
```

Exercise 7 — Character Frequency Counter

Input a string of characters. - Use a HashMap to count how many times each character appears in the string. - Output: a HashMap where the key is a character and the value is the number of times it appears.

Exercise 8 — Most Frequent Element in an Array

Input an array of integers. - Use a HashMap to find the element that appears most frequently in the array. - Output: the most frequent element and how many times it occurs.

Exercise 9 — Simple LRU Cache (Bonus / Advanced)

Requirements: - Implement a cache using the Least Recently Used (LRU) eviction strategy. - Use a HashMap together with a doubly linked list (LinkedList) to manage storage and access order. - Input: the cache capacity and a sequence of access requests. - Output: the state of the cache after each access.

Exercise 10 — Student Grade Lookup (HashTable)

Requirements: - Implement a HashMap (or Hashtable) to store student names as keys and their grades as values. - Add at least 5 students to the map. - Allow the user to search for a student's grade based on a name entered from the keyboard; if not found, print "Not found".

Exercise 11 — Build a Tree from Categories and Subcategories

Write a function to convert a list of arrays (representing categories and subcategories) into a tree structure.

Example:

```
String[][] categories = {
    {"Electronics", "Phones"},
    {"Electronics", "Laptops"},
    {"Home", "Furniture"},
    {"Home", "Appliances"}
};
// Convert to:
// Electronics
//     - Phones
//     - Laptops
// Home
//     - Furniture
//     - Appliances

void printTree(TreeNode node,
String indent) {
System.out.println(indent +
node.data);
  for (TreeNode child :
node.children) {
    printTree(child, indent + "
");
  }
}
// Example usage:
printTree(root, "");
```

Exercise 13 — Binary Search Tree (BST) with Removal

Write a program to create a Binary Search Tree (BST) from the list of integers [45, 23, 67, 12, 34, 89, 50].

- Print the tree using Pre-order Traversal.
- Remove the element 34 from the tree and reprint the tree using Pre-order Traversal.

Exercise 14 — Balanced Binary Search Tree

Create a Balanced Binary Search Tree (BST) from the sorted list of integers [10, 20, 30, 40, 50, 60, 70, 80].

(You may sort and build a balanced BST from a sorted array by repeatedly picking the middle element as the root.)

- Print the tree using In-order Traversal (the result should be an increasing sequence).
- Optional bonus: Also print the Pre-order and Post-order traversals.

Exercise 14 — Balanced Binary Tree

Create a Balanced Binary Tree from the list of integers [50, 30, 70, 20, 40, 60, 80]. Print the tree using In-order Traversal.